

Aula 09 — Máquinas de Vetores de Suporte (SVM)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §8.2 e §4.6; ISLP cap. 9

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- definir hiperplano separador e o **classificador de margem máxima**;
- identificar os **vetores de suporte** e o papel da margem;
- generalizar para dados não separáveis com o **classificador de vetor de suporte** (*soft margin*) e o hiperparâmetro C ;
- explicar o **truque do kernel** a partir da forma dual;
- usar *kernels* (polinomial, gaussiano/RBF) para fronteiras não lineares e entender γ ;
- reconhecer a SVM como minimização da *perda* hinge regularizada e sua relação com a regressão logística; tratar o caso multiclasse.

Em sala

Mudança de perspectiva em relação à Aula 07: lá modelávamos $\mathbb{P}(Y \mid \mathbf{x})$; a SVM ataca a fronteira *geometricamente*, buscando a separação “mais folgada”. É um dos métodos mais elegantes do curso — liga geometria, otimização e os RKHS (§4.6 do [AME]). Pergunta de abertura: “se há infinitos hiperplanos que separam as classes, qual é o melhor?”

1 Hiperplanos e o classificador de margem máxima

Codifique as classes como $y \in \{-1, +1\}$. Um **hiperplano** em $\mathbb{R}^p$ é o conjunto $\{\mathbf{x} : f(\mathbf{x}) = 0\}$ com $f(\mathbf{x}) = \beta_0 + \beta^\top \mathbf{x}$. Ele separa o espaço em dois lados conforme o sinal de f . Dizemos que os dados são *linearmente separáveis* se existe f tal que

$$y_i f(\mathbf{X}_i) > 0 \quad \text{para todo } i. \quad (1)$$

A magnitude $|f(\mathbf{x})|$ (com $\|\beta\| = 1$) é a *distância* de $\mathbf{x}$ ao hiperplano.

Ideia central

Quando há *vários* hiperplanos separadores, a SVM escolhe o de **maior margem**: a maior distância mínima aos pontos. A intuição: deixar a maior “folga” possível entre as classes tende a generalizar melhor.

Definição 1.1 (Classificador de margem máxima). Procura-se o hiperplano que maximiza a margem M :

$$\max_{\beta_0, \beta} M \quad \text{s.a.} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad y_i f(\mathbf{X}_i) \geq M \quad \forall i.$$

A restrição $\|\beta\| = 1$ torna os hiperplanos comparáveis (assim $y_i f(\mathbf{X}_i)$ é a distância com sinal). As observações que *atingem* a margem ($y_i f(\mathbf{X}_i) = M$) são os **vetores de suporte**.

Atenção

Apenas os vetores de suporte determinam o hiperplano: mover um ponto longe da margem não muda nada. Por outro lado, a formulação *exige* separabilidade perfeita e é **muito sensível** a pequenas mudanças nos dados — uma única observação pode alterar drasticamente a solução. Precisamos relaxá-la.

2 O classificador de vetor de suporte (*soft margin*)

Para lidar com dados não separáveis (e ganhar robustez), permitimos que algumas observações violem a margem, introduzindo *folgas* $e_i \geq 0$:

$$\max_{\beta_0, \beta, e} M \quad \text{s.a.} \quad \|\beta\| = 1, \quad y_i f(\mathbf{X}_i) \geq M(1 - e_i) \quad \forall i, \quad e_i \geq 0, \quad \sum_{i=1}^n e_i \leq C. \quad (2)$$

A folga e_i mede o quanto a i -ésima observação viola a margem: $e_i > 0$ significa dentro da margem; $e_i > 1$ significa do *lado errado* do hiperplano (erro de classificação).

Ideia central

$C \geq 0$ é o **tuning parameter** (orçamento de violações), escolhido por validação cruzada (Aula 03):

- *C grande*: tolera muitas violações $\Rightarrow$ margem larga, mais vetores de suporte, modelo mais *rígido* (alto viés, baixa variância);
- *C pequeno*: poucas violações $\Rightarrow$ margem estreita, modelo mais *flexível* (baixo viés, alta variância).

É o balanço viés-variância da Aula 01 de novo. (Atenção: a convenção de C no `scikit-learn` é *invertida* — lá C grande penaliza mais as violações.)

3 O truque do *kernel*

A chave que torna a SVM poderosa: a solução depende dos dados *apenas através de produtos internos*. Pode-se mostrar que a função ótima se escreve

$$f(\mathbf{x}) = \beta_0 + \sum_{k=1}^n \alpha_k \langle \mathbf{x}, \mathbf{X}_k \rangle, \quad \langle \mathbf{x}, \mathbf{X}_k \rangle = \sum_{j=1}^p x_j x_{k,j}, \quad (3)$$

e que o cálculo dos α_k também só requer os produtos internos $\langle \mathbf{X}_i, \mathbf{X}_k \rangle$ entre as observações. Crucialmente, $\alpha_k = 0$ para todos os pontos que *não* são vetores de suporte; logo

$$f(\mathbf{x}) = \beta_0 + \sum_{k \in S} \alpha_k \langle \mathbf{x}, \mathbf{X}_k \rangle,$$

onde S é o conjunto de vetores de suporte.

Ideia central

Como tudo depende só de produtos internos, podemos **substituir** $\langle \mathbf{x}, \mathbf{X}_k \rangle$ por um *kernel* $K(\mathbf{x}, \mathbf{X}_k)$ — que corresponde a um produto interno num espaço de *features* de dimensão muito maior (ou infinita), sem *nunca* computar esse espaço explicitamente. Isso é o **truque do kernel**: fronteiras não lineares no espaço original a custo de fronteiras lineares num espaço ampliado.

Kernels de Mercer comuns (cf. [AME], §4.6):

Kernel	$K(\mathbf{x}, \mathbf{x}')$
Linear	$\langle \mathbf{x}, \mathbf{x}' \rangle$
Polinomial	$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^p$
Gaussiano (RBF)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$

No kernel gaussiano, $\gamma > 0$ é um *tuning parameter*: γ grande $\Rightarrow$ influência local, fronteiras muito flexíveis (risco de superajuste); γ pequeno $\Rightarrow$ fronteiras suaves. Escolhem-se C e γ juntos por validação cruzada.

4 SVM como perda *hinge* regularizada

Há uma leitura que conecta a SVM a tudo o que vimos. Dado um kernel de Mercer K com RKHS $\mathcal{H}_K$ ([AME], §4.6), o classificador da SVM minimiza

$$\min_{g \in \mathcal{H}_K} \sum_{k=1}^n \underbrace{(1 - y_k g(\mathbf{X}_k))_+}_{\text{perda } hinge} + \lambda \|g\|_{\mathcal{H}_K}^2, \quad (4)$$

onde $(u)_+ = \max(u, 0)$. Ou seja, a SVM é *ajuste de perda + regularização* (Aula 02), com a **perda *hinge*** $L(y, g) = (1 - yg)_+$.

Observação 4.1. A perda *hinge* é **zero** quando $yg(\mathbf{x}) \geq 1$, isto é, quando o ponto está do lado certo e *folgadoamente* longe da margem. Só os pontos próximos ou violando a margem contribuem — e são exatamente os *vetores de suporte*. No caso separável, (4) reencontra a margem máxima $(1/\|g\|)$.

4.1 Relação com a regressão logística

A SVM (*hinge*) e a regressão logística (perda logística $\log(1 + e^{-yg})$) são ambas “perda de margem + regularização ℓ_2 ”. As perdas são parecidas, mas:

- a *hinge* é exatamente zero longe da margem $\Rightarrow$ solução *esparsa* em vetores de suporte; a logística nunca é zero;
- a logística entrega **probabilidades** calibradas (Aula 08); a SVM entrega só a fronteira (probabilidades exigem pós-processamento, ex. *Platt scaling*).

Regra prática: classes bem separadas $\rightarrow$ SVM costuma ir muito bem; quando se quer $\mathbb{P}(Y | \mathbf{x})$, prefira a logística.

5 Mais de duas classes

A SVM é intrinsecamente binária. Para $|\mathcal{C}| > 2$:

Um-contra-um (OvO)

treina uma SVM para cada par de classes ($\binom{K}{2}$ classificadores) e classifica por votação. Padrão do `scikit-learn`.

Um-contra-todos (OvA)

treina K SVMs (cada classe vs. o resto) e escolhe a de maior $f(\mathbf{x})$.

6 Prática em Python

```

1 from sklearn.svm import SVC
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 from sklearn.model_selection import GridSearchCV
5
6 # SVM com kernel RBF; padronizar e ESSENCIAL (metodo baseado em distancia)
7 pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
8 grade = {"svc__C": [0.1, 1, 10, 100],
9          "svc__gamma": [0.001, 0.01, 0.1, 1]}
10 busca = GridSearchCV(pipe, grade, cv=5, scoring="accuracy")
11 busca.fit(X_tr, y_tr)
12 print("melhores parametros:", busca.best_params_)
13
14 # Para probabilidades: SVC(probability=True) (usa Platt scaling, mais lento)

```

Os notebooks Aula prática 09 e Exemplo - SVM em dados artificiais ilustram o efeito de C , γ e da escolha do kernel sobre a fronteira de decisão.

Atenção

SVM com kernel RBF é **sensível à escala** das covariáveis (depende de distâncias) — sempre padronize dentro do *pipeline* (Aula 06).

Resumo

- **Margem máxima:** hiperplano separador de maior folga; definido pelos *vetores de suporte*; exige separabilidade e é frágil.
- **Soft margin** (2): folgas e_i e orçamento C (tuning) controlam o balanço viés–variância.
- **Truque do kernel:** a solução só depende de produtos internos (3); trocá-los por K dá fronteiras não lineares (polinomial, RBF com γ).
- SVM = perda *hinge* + regularização (4) (RKHS, [AME] §4.6); esparsa em vetores de suporte.
- Relação com a logística (perda de margem + ℓ_2); SVM não dá probabilidades de imediato.
- Multiclasse por OvO/OvA; *padronize* sempre.

Leitura recomendada. [AME] §8.2 (SVM) e §4.6 (RKHS e truque do kernel); [ISLP] Capítulo 9 (§9.1 margem máxima, §9.2 classificador de vetor de suporte, §9.3 SVM com kernels, §9.4 multiclasse, §9.5 relação com a logística).